Task 02 — Professional API Design

Implement the following in your **Spring Boot REST API**:

1. Naming Standards

Use standard REST naming conventions.

Operation	HTTP Method	Endpoint
Get all products	GET	`/api/products`
Get product by ID	GET	`/api/products/{id}`
Create product	POST	`/api/products`
Update product	PUT	`/api/products/{id}`
Delete product	DELETE	`/api/products/{id}`

Use plural nouns such as products, orders, and users.

2. Create DTOs

Create a package:

src/main/java/com/example/demo/dto

Create ProductRequestDTO.java:

```
package com.example.demo.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;

public class ProductRequestDTO {

    @NotBlank(message = "Product name is required")
    private String name;
```

```
    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    private Double price;

    @NotBlank(message = "Category is required")
    private String category;

    public ProductRequestDTO() {
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Double getPrice() {
        return price;
    }

    public void setPrice(Double price) {
        this.price = price;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }
}
```

Create `ProductResponseDTO.java`:

```
package com.example.demo.dto;

public class ProductResponseDTO {

    private Long id;
    private String name;
    private Double price;
    private String category;

    public ProductResponseDTO() {
    }

    public ProductResponseDTO(Long id, String name, Double price, String category) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.category = category;
    }

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public Double getPrice() {
        return price;
    }

    public String getCategory() {
        return category;
    }
}
```

3. Validation

Add the validation dependency if it is not already present.

For Maven:

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Then in the Controller use:

```
@PostMapping
public ResponseEntity<ProductResponseDTO> createProduct(
      @Valid @RequestBody ProductRequestDTO request) {

   // create product
}
```

Examples of validation:

```
@NotBlank
private String name;

@Positive
private Double price;

@NotNull
private String category;
```

4. Pagination

Use Spring Data's `Pageable`.

```
@GetMapping
public Page<Product> getProducts(Pageable pageable) {
   return productRepository.findAll(pageable);
}
```

Example request:

```
GET /api/products?page=0&size=10
```

Here:

- `page=0` → first page
- `size=10` → 10 products per page

5. Sorting

Allow the client to specify the sort field:

```
GET /api/products?sort=price,asc
```

Descending:

```
GET /api/products?sort=price,desc
```

Multiple sorting:

```
GET /api/products?sort=category,asc&sort=price,desc
```

Controller:

```
@GetMapping
public Page<Product> getProducts(Pageable pageable) {
   return productRepository.findAll(pageable);
```

}

Spring Boot automatically handles the `Pageable` parameters.

6. Filtering

Add a category filter:

```
GET /api/products?category=Electronics
```

Repository:

```
List<Product> findByCategory(String category);
```

Controller:

```
@GetMapping("/filter")
public List<Product> filterProducts(
      @RequestParam String category) {

   return productRepository.findByCategory(category);
}
```

Final Structure

Your project should look approximately like:

```
src/main/java/com/example/demo
│
├── controller
│   └── ProductController.java
│
├── service
│   └── ProductService.java
│
├── repository
│   └── ProductRepository.java
│
├── entity
│   └── Product.java
│
└── dto
    ├── ProductRequestDTO.java
    └── ProductResponseDTO.java
```

This completes Task 02 — Professional API Design for the Spring Boot REST API.